Les variables et type de données

Tous droits réservés à STUDI - Reproduction interdite

Table des matières

I. Contexte 3

II. Les variables 3

A. Les variables 3

III. Les types de données en Python 4

A. Qu'est-ce qu'un type de données et pourquoi est-il important ? 4

B. Les principaux types de données 5

C. Exercice : Quiz 7

IV. La portée des variables et notion de référence 8

A. Qu'est-ce que la portée des variables ? 8

B. Notion de référence en Python 9

V. Essentiel 10

VI. Auto-évaluation 10

A. Exercice 10

B. Test 10

Solutions des exercices 11

Tous droits réservés à STUDI - Reproduction interdite

I. Contexte

Durée : 1 h

Environnement de travail : PC ou Mac avec 8 Go de ram minimum

Prérequis : aucun

Contexte

Comprendre comment les variables Python fonctionnent est une étape fondamentale, car elles permettent de stocker, manipuler et organiser les données dans un programme.

Ce cours vise à expliquer les bases des variables en Python, leur fonctionnement, les différents types de données, ainsi que des notions plus avancées comme la portée des variables et le concept de référence.

Nous allons explorer ces concepts à travers des exemples concrets, des bonnes pratiques et des exercices pour garantir une compréhension approfondie.

Objectifs pédagogiques : comprendre comment fonctionnent les variables en Python, leur portée et la notion de référence en mémoire, afin d'écrire un code clair et structuré.

II. Les variables

A. Les variables

Les variables sont l'un des concepts fondamentaux en programmation, présents dans la quasi-totalité des langages. Sans elles, il serait impossible d'écrire un programme, car elles permettent de stocker et de manipuler des données.

Exemple

Une variable peut être vue comme une boîte dans laquelle l'ordinateur conserve une valeur unique. Si une donnée doit être réutilisée ou modifiée au cours du programme, il est essentiel de l'enregistrer dans une variable, un peu comme si l'on étiquetait une boîte pour mieux retrouver son contenu.

En Python, une variable est représentée par un mot qui commence par une lettre et peut contenir des lettres, des chiffres et le caractère underscore (_). Par convention, les noms de variables, d'attributs et de fonctions sont écrits en minuscules, avec éventuellement des chiffres, mais sans espaces ni caractères spéciaux. Un bon nom de variable doit refléter son contenu de manière explicite.

Par exemple, si vous étiquetez toutes vos boîtes de déménagement avec le mot trucs, il vous sera difficile de retrouver ce que vous cherchez.

Le nommage des variables en Python est également sensible à la **casse**, ce qui signifie que spam, SPAM, Spam et sPaM sont considérés comme quatre variables distinctes. Il est donc recommandé d'adopter une convention claire et cohérente, comme l'usage du `snake_case`, où les mots sont séparés par un underscore (_), facilitant ainsi la lisibilité du code.

Exemple

```
1 look_like_this
```

L'affectation des variables et modification de sa valeur

En Python, une variable stocke une valeur à l'aide d'une instruction d'affectation. Cette instruction suit une structure simple : **un nom de variable (le conteneur)**, suivi d'**un signe égal (=) (l'opérateur d'affectation)**, et enfin **une valeur (le contenu)** à stocker.

Exemple

```
1 ma_variable = 42
```

Ici, la variable `ma_variable` est créée et contient la valeur 42.

Modification de la valeur d'une variable

Une variable peut être réassignée plusieurs fois au cours d'un programme. À chaque nouvelle affectation, l'ancienne valeur est remplacée et définitivement oubliée.

Exemple

```
1 legumes = 3
2 print(legumes) # Affiche 3
3
4 legumes = 5
5 print(legumes) # Affiche 5
```

Dans cet exemple, `legumes` commence avec la valeur 3, mais après l'affectation `legumes = 5`, la valeur précédente est effacée et seule la nouvelle valeur est conservée.

Cette flexibilité permet de manipuler les variables librement, mais il est essentiel de bien suivre leur évolution pour éviter toute confusion dans le programme.

III. Les types de données en Python

A. Qu'est-ce qu'un type de données et pourquoi est-il important ?

Dans un programme Python, chaque valeur appartient à une catégorie spécifique, appelée type de données. Un type définit la nature d'une donnée, les opérations possibles sur celle-ci, ainsi que la manière dont elle est stockée en mémoire. Comprendre les types de données est essentiel, car ils influencent directement la manière dont Python manipule les variables et les calculs.

Les types sont importants car ils permettent à Python de mieux gérer la mémoire en allouant uniquement l'espace nécessaire aux données. Ils évitent également des erreurs courantes en interdisant certaines opérations illogiques.

Par exemple, si l'on essaie d'additionner un texte et un nombre, Python renverra une erreur, car ces types ne sont pas compatibles sans conversion explicite.

Enfin, chaque type propose des outils spécialisés qui facilitent la programmation. Une chaîne de caractères, par exemple, possède des méthodes comme `.upper()` pour mettre le texte en majuscules, tandis qu'un nombre permet d'effectuer des calculs mathématiques. Ces caractéristiques font des types une notion fondamentale à maîtriser pour écrire du code clair et efficace.

Dans un langage comme Python, le type d'une variable est déterminé automatiquement lors de l'affectation et peut changer en cours d'exécution. C'est ce que l'on appelle un **typage dynamique**. Contrairement à d'autres langages où l'on doit déclarer explicitement le type des variables (comme en C ou Java), **Python ajuste automatiquement le type en fonction de la valeur assignée**. Cela apporte une grande flexibilité dans l'écriture du code.

Tous droits réservés à STUDI - Reproduction interdite

Exemple

```
ma_variable = 10
print(type(ma_variable)) # <class ‘int>

ma_variable = Python”
print(type(ma_variable)) # <class ‘str’t>
```

Dans ce cas, `ma_variable` commence comme un entier (int), puis devient une chaîne de caractères (str) sans qu’aucune déclaration explicite ne soit nécessaire. Python adapte donc le type de la variable à son contenu.

B. Les principaux types de données

En Python, chaque valeur appartient à un type spécifique, qui détermine les opérations possibles et la manière dont la donnée est manipulée par le programme. Il est essentiel de bien comprendre ces types pour éviter des erreurs et optimiser son code.

Le tableau suivant résume les principaux types de données en Python :

Type de données	Nom en Python	Exemples	Description
Entier	int	-2, 0, 42, 1000	Nombre entier, positif ou négatif, sans partie décimale
Flottant	float	3.14, -1.25, 42.0	Nombre à virgule flottante (avec partie décimale)
Chaîne de caractères	str	'Bonjour', "Python", '42'	Suite de caractères entourée de guillemets
Booléen	bool	True, False	Valeur logique (Vrai ou Faux)
Liste	list	[1, 2, 3], ['a', 'b', 'c']	Collection ordonnée et modifiable d’éléments
Tuple	tuple	(1, 2, 3), ('x', 'y')	Collection ordonnée et non modifiable d’éléments
Dictionnaire	dict	{"nom": "Alice", "âge": 25}	Collection de paires clé-valeur
Ensemble	set	{1, 2, 3}, {'a', 'b', 'c'}	Collection non ordonnée et sans doublon

Les nombres en Python

Les nombres en Python sont divisés en deux catégories principales :

- **Les entiers (int)** sont des nombres sans partie décimale, comme 0, 42 ou -100. Ils sont utilisés pour compter ou représenter des quantités précises.
- **Les nombres à virgule flottante (float)** possèdent une partie décimale, comme 3.14, -1.25 ou 42.0. Ils sont utilisés pour les mesures et les calculs nécessitant une précision décimale.

Tous droits réservés à STUDI - Reproduction interdite

Exemple

```
1 entier = 10
2 flottant = 3.14
3 attention_float = 42.0  # puisqu'il y a un point ce n'est plus un entier mais un flottant
4
5 print(type(entier))    # <class 'int'>
6 print(type(flottant))  # <class 'float'>
7 print(type(attention_float))  # <class 'float'>
```

Les chaînes de caractères (str)

Les chaînes de caractères sont des textes entourés de guillemets simples (') ou doubles ("). Elles permettent de manipuler du texte, comme des noms, des messages ou du contenu utilisateur.

```
1 texte1 = "Bonjour"
2 texte2 = 'Python'
3 texte3 = "42"  # Attention : c'est une chaîne, pas un nombre !
```

Opérations possibles sur les chaînes

Sur des chaînes de caractères il est possible de les regrouper dans une même variable, on appelle cela la concaténation. Pour cela il suffit d'ajouter le symbole + .

Exemple

```
1 nom = "Alice"
2 message = "Bonjour, " + nom
3 print(message)  # Bonjour, Alice
```

La répétition peut être utile et s'utilise avec le simple * , l'opération doit être entre parenthèses :

```
1 print("Hello" * 3)  # HelloHelloHello
```

Voici d'autres méthodes souvent utilisés pour manipuler les chaînes de caractères :

Méthode	Utilisation	Description
upper()	"python".upper()	Convertit la chaîne en majuscules → "PYTHON"
lower()	"PYTHON".lower()	Convertit la chaîne en minuscules → "python"
replace()	"bonjour".replace("o", "a")	Remplace un caractère ou une sous-chaîne par une autre → "banjour"

Les booléens (bool)

Un booléen représente une valeur logique : **True (Vrai)** ou **False (Faux)**. Ils sont souvent utilisés dans les conditions **(if, while)**.

Tous droits réservés à STUDI - Reproduction interdite

Exemple

```
1 est_adulte = True
2 a_devoirs = False
3
4 if est_adulte:
5     print("Vous pouvez entrer.")  # Ce message s'affichera
```

Les collections : listes (list), tuples (tuple), ensembles (set) et dictionnaires (dict)

Python permet de stocker plusieurs valeurs ensemble grâce aux collections, voici un tableau descriptif de ces collections :

Type de collection	Nom en Python	Exemples	Caractéristiques
Liste	list	[1, 2, 3], ['a', 'b']	Collection ordonnée et modifiable
Tuple	tuple	(1, 2, 3), ('x', 'y')	Collection ordonnée mais non modifiable
Ensemble	set	{1, 2, 3}, {'a', 'b'}	Collection non ordonnée, sans doublon
Dictionnaire	dict	{"nom": "Alice", "âge": 25}	Collection de paires clé-valeur

En Python, chaque valeur appartient à un type de données qui détermine la manière dont elle peut être utilisée.

- Les nombres **(int, float)** sont utilisés pour les **calculs**.
- Les chaînes **(str)** permettent de manipuler du **texte**.
- Les booléens **(bool)** servent à la **logique conditionnelle**.
- Les collections **(list, tuple, set, dict)** permettent de **stocker et organiser** plusieurs éléments.

C. Exercice : Quiz

[solution n°1 p.13]

Question 1

Lequel de ces types est valide en Python ?

- ○ Integer
- ○ Collection
- ○ Itérable

Question 2

Les variables Python sont sensibles à la casse.

- ○ Vrai
- ○ Faux

Tous droits réservés à STUDI - Reproduction interdite

Question 3

On peut concaténer une variable de type **str** et une variable de type **integer**.

- Vrai
- Faux

Question 4

Par quoi est déterminé le type de la variable ?

- Le conteneur
- Le nom du type
- Le contenu

Question 5

La norme snake_case est acceptée pour donner un nom à une variable Python.

- Vrai
- Faux

IV. La portée des variables et notion de référence

A. Qu'est-ce que la portée des variables ?

En Python, chaque variable a une portée (*scope* en anglais), c'est-à-dire un contexte dans lequel elle peut être utilisée. Comprendre la portée d'une variable est essentiel pour éviter les erreurs et organiser son code efficacement.

Lorsque vous déclarez une variable dans un programme, elle n'est pas accessible partout. En Python elles ont soit une portée globale soit locale.

Portée locale et globale

En Python, on distingue deux types de portée :

- Portée globale (*global scope*) : une variable déclarée en dehors d'une fonction peut être utilisée partout dans le programme.
- Portée locale (*local scope*) : une variable déclarée dans une fonction est accessible uniquement dans cette fonction.

Exemple

```python
x = 10  # Variable globale car elle est déclarée en dehors d'une fonction

def afficher_x():
    print(x)  #   Accessible car x est globale

afficher_x()  # Affiche 10

def test():
    y = 20  # Variable locale car elle est déclarée dans la fonction
    print(y)

test()  # Affiche 20
```


Tous droits réservés à STUDI - Reproduction interdite

```
print(y)  # Erreur : y n'existe que dans la fonction
```

Complément **Modifier une variable globale dans une fonction**

Par défaut, une fonction ne peut pas modifier une variable globale directement. Pour le faire, il faut utiliser le mot-clé `global`

Exemple

```
x = 5  # Variable globale

def modifier():
    global x
    x = 10  # On modifie la variable globale

modifier()
print(x)  # Affiche 10
```

Attention

L'utilisation de `global` est généralement déconseillée, car elle peut rendre le code difficile à comprendre et à déboguer.

Notez également que si une variable globale et une variable locale portent le même nom, Python considère uniquement la variable locale à l'intérieur de la fonction.

Exemple

```
x = 5  # Variable globale

def exemple():
    x = 10  # Variable locale
    print(x)  # Affiche 10

exemple()
print(x)  # Affiche 5 (la variable globale n'a pas changé)
```

La variable x dans `exemple()` n'affecte pas la variable x globale.

B. Notion de référence en Python

En Python, les variables ne contiennent pas directement les valeurs mais des ***références*** à des objets en mémoire. Cela signifie que plusieurs variables peuvent faire référence au même objet.

Exemple

```
a = [1, 2, 3]  # a fait référence à une liste en mémoire
b = a  # b fait référence au même objet que a

b.append(4)  # Modification de la liste
print(a)  # [1, 2, 3, 4], car a et b référencent le même objet
```

Ici, a et b pointent vers le même objet en mémoire, donc modifier **b** impacte aussi **a**.

Si on ne souhaite pas avoir une influence sur la variable de référence il est nécessaire de copier un objet sans partager la référence

Tous droits réservés à STUDI - Reproduction interdite

Si on veut créer une copie indépendante, on utilise `copy()` en n'oubliant pas d'importer Copy dans le code

```
import copy
c = [1, 2, 3]
d = copy.copy(c)  # Crée une nouvelle liste indépendante

d.append(5)
print(c)  # [1, 2, 3], car d est une copie indépendante
```

V. Essentiel

Dans ce cours, nous avons exploré les concepts fondamentaux des variables en Python : leur définition, leur portée et la notion de référence en mémoire. Nous avons vu que Python utilise un **typage dynamique**, ce qui permet aux variables de changer de type au cours de l'exécution.

La compréhension des variables et de leur portée est essentielle pour éviter des erreurs courantes et écrire un code plus propre et efficace. Enfin, la notion de référence en mémoire est un aspect crucial à maîtriser pour bien manipuler les objets mutables en Python.

Maîtriser ces bases vous permettra d'être plus à l'aise avec la programmation Python et de mieux comprendre les concepts avancés que nous aborderons dans la suite du cours.

VI. Auto-évaluation

A. Exercice

Vous travaillez dans un magasin et devez suivre l'inventaire des articles en stock. Chaque article est associé à une quantité disponible.

Question 1

[solution n°2 p.14]

Créer une variable `stock_pommes` contenant le nombre de pommes en stock (ex : 50). Un client achète 5 pommes. Mettez à jour la variable `stock_pommes`. Afficher le stock mis à jour.

Copier une liste sans modifier l'originale

Un programme manipule une liste d'invités à une soirée. Vous devez faire une copie de cette liste pour effectuer des modifications sans altérer l'originale.

Question 2

[solution n°3 p.14]

Créer une liste `invites` contenant des noms puis copier cette liste sans partager la référence. Ajouter un invité à la copie sans modifier l'originale et enfin afficher les deux listes pour vérifier que l'originale n'a pas été modifiée.

B. Test

Exercice 1 : Quiz

[solution n°4 p.14]

Question 1

Qu'est-ce que la portée d'une variable ?

- ○ Sa valeur
- ○ La plage d'instructions dans le code où elle est accessible
- ○ Son nom

Question 2

Tous droits réservés à STUDI - Reproduction interdite

Quels sont les deux types de portées ?

- ☐ Portée générale
- ☐ Portée locale
- ☐ Portée globale

Question 3

Une variable fait référence à un objet.

- ○ Vrai
- ○ Faux

Question 4

Comment pouvez-vous forcer une variable dans une fonction à se référer à la variable globale ?

- ○ Avec l'instruction « *global* »
- ○ Avec l'instruction « *return* »
- ○ Avec l'instruction « *def* »

Question 5

Quel sera le résultat du code Python suivant ?

```
1 def f1():
2     x=15
3     print(x)
4 x=12
5 f1()
```

- ○ Error
- ○ 12
- ○ 15
- ○ 1512

Solutions des exercices

Tous droits réservés à STUDI - Reproduction interdite

studi

Exercice p. 7 Solution n°1

Question 1

Lequel de ces types est valide en Python ?

- (●) Integer
- ( ) Collection
- ( ) Itérable

C'est le type integer qui est valide en Python, les autres types sont issues de Java.

Question 2

Les variables Python sont sensibles à la casse.

- (●) Vrai
- ( ) Faux

Les noms de variables sont sensibles à la casse, ce qui signifie que « *spam* », « *SPAM* », « *Spam* » et « *sPaM* » sont quatre variables différentes.

Question 3

On peut concaténer une variable de type **str** et une variable de type **integer**.

- ( ) Vrai
- (●) Faux

On ne peut pas les concaténer car Python est un langage à fort typage dynamique.

Question 4

Par quoi est déterminé le type de la variable ?

- ( ) Le conteneur
- ( ) Le nom du type
- (●) Le contenu

Le type de la variable est déterminé par le contenu comme dans tous les langages typés dynamiquement.

Question 5

La norme snake_case est acceptée pour donner un nom à une variable Python.

- (●) Vrai
- ( ) Faux

La norme snake_case est valide comme nom de variable.

Tous droits réservés à STUDI - Reproduction interdite

p. 10 Solution n°2

```
stock_pommes = 50  # Initialisation du stock
achat_client = 5  # Nombre de pommes achetée

stock_pommes -= achat_client  # Mise à jour du stock

print("Stock restant de pommes :", stock_pommes)  # Affiche 45
```

p. 10 Solution n°3

```
import copy

invites = ["Alice", "Bob", "Charlie"]  # Liste originale
invites_copie = copy.copy(invites)  # Faire une copie indépendante

invites_copie.append("David")  # Ajouter un invité à la copie

print("Liste originale :", invites)  # Affiche ['Alice', 'Bob', 'Charlie']
print("Liste modifiée :", invites_copie)  # Affiche ['Alice', 'Bob', 'Charlie', 'David']
```

Exercice p. 10 Solution n°4

Question 1

Qu'est-ce que la portée d'une variable ?

- ○ Sa valeur
- ◉ La plage d'instructions dans le code où elle est accessible
- ○ Son nom

La portée d'une variable, c'est l'ensemble des lignes de codes où on pourra accéder à cette variable.

Question 2

Quels sont les deux types de portées ?

- ☐ Portée générale
- ☑ Portée locale
- ☑ Portée globale

La portée globale correspond à l'ensemble du programme tandis qu'une portée locale correspond à la fonction dans laquelle est déclarée une variable locale.

Question 3

Une variable fait référence à un objet.

- ◉ Vrai
- ○ Faux

Tous droits réservés à STUDI - Reproduction interdite

Une variable est une référence nommée vers un objet.

Question 4

Comment pouvez-vous forcer une variable dans une fonction à se référer à la variable globale ?

- (●) Avec l'instruction « *global* »
- ( ) Avec l'instruction « *return* »
- ( ) Avec l'instruction « *def* »

L'instruction « *global* » sert à déclarer et utiliser une variable globale au sein d'une fonction, et donc à ne pas se référer à une variable locale.

Question 5

Quel sera le résultat du code Python suivant ?

```
1 def f1():
2     x=15
3     print(x)
4 x=12
5 f1()
```

- ( ) Error
- ( ) 12
- (●) 15
- ( ) 1512

La variable x est une variable locale. Sa portée correspond à l'ensemble des lignes de code de la fonction f1(). Donc, l'instruction print(x), appartenant à cette même portée affichera 15 qui est la valeur affectée à la variable locale x.

Tous droits réservés à STUDI - Reproduction interdite

Tous droits réservés à STUDI - Reproduction interdite